



INDIA.RUNS

Build what next India runs on

Team Name : Binary Pulse

Team Leader Name : Bhoomi Chowksey

Problem Statement :

Recruiters sift through hundreds of profiles using keyword-matching ATS tools that miss great candidates, introduce bias, and waste time on poor-fit shortlists. Build an AI system that ranks candidates the way a great recruiter would.

Solution Overview

- HireScore is a deterministic candidate ranking portal (Python + Gradio) that scores each candidate across 5 weighted dimensions — Skills (35%), Career (35%), Experience (15%), Location (10%), Education (5%) — then applies a Behavioral Multiplier (0.3×–1.2×). Final score is 0–100 with full per-dimension breakdowns. Every decision is auditable, no black box. Live demo: huggingface.co/spaces/bhoomichowksey/redrob-ranker-demo
- Unlike keyword-matching ATS tools, HireScore evaluates skill depth (JD-weight × proficiency × duration × assessment), career pedigree (product co. 1.0 vs consulting 0.28), a non-linear experience curve (6–8 yrs = 1.0), location + notice period fit, institution tier, and behavioral signals (open-to-work, response rate, GitHub activity) as a score multiplier. No LLM, no hallucinations — pure deterministic arithmetic.

JD Understanding & Candidate Evaluation

- JD requirements are encoded in a JD_SKILLS dictionary of 20 ML/Data skills with importance weights: python (0.95), retrieval/RAG (0.88–0.90), ML/NLP (0.85–0.90), pytorch (0.85), tensorflow (0.80), sql (0.70), fastapi (0.65), docker (0.60). Skills absent from the dictionary contribute 0. Company type is determined by PRODUCT_COS and CONSULT lists; role fit by ML/AI/NLP/LLM keywords in the title.
- HireScore evaluates 6 signals beyond keywords: (1) Skill depth — proficiency (1–5) and months per skill, not just presence. (2) Career pedigree — product co. (1.0) vs consulting (0.28) vs other (0.60). (3) Experience curve — 6–8 yr sweet spot = 1.0, tapering below 4 and above 14 yrs. (4) Location — 9 top tech cities + notice period (0 days = 1.0, >60 days = 0.20). (5) Education — IIT/BITS/NIT/IIIT/DTU tier = 1.0, others 0.55. (6) Behavioral — open-to-work, response rate, GitHub activity applied as 0.3–1.2× multiplier.

Ranking Methodology

- Recruiters enter up to 5 candidates via Gradio tabs with fields: name, years of experience, city, notice period, company, role title, institution, open-to-work, response rate, GitHub score, and skills (Name, Proficiency 1-5, Months). The engine runs 6 pure Python functions, combines weighted scores, applies the behavioral multiplier, multiplies by 100, and outputs a ranked list with medal display and CSV export.
- No external ML model is used. All scoring is deterministic: JD_SKILLS weight dict for skill relevance; proficiency/5 normalization; $\min(\text{months}/36, 1.0)$ for duration; hard-coded company lists (PRODUCT_COS, CONSULT) for career; piecewise function for experience curve; city allowlist + notice period for location; institution keyword match for education. Behavioral multiplier = $\min(0.3 + 0.9 \times \text{composite}, 1.2)$.
- Final Score = $(0.35 \times \text{Skills} + 0.35 \times \text{Career} + 0.15 \times \text{Exp} + 0.10 \times \text{Loc} + 0.05 \times \text{Edu}) \times \text{Behavioral Multiplier} \times 100$. Skills = $\text{sum}(\text{per_skill_score}) / \text{sum}(\text{jd_weights})$, where

Explainability & Data Validation

- Every ranking is fully transparent. The output shows each dimension's score with an ASCII progress bar, all sub-scores (`company_score`, `role_score`, `city_score`, `notice_score`), the behavioral multiplier value, and the weighted sum before and after the multiplier. A CSV export covers all scores. Recruiters can see exactly why Candidate A outranked Candidate B — no guesswork.
- Zero hallucinations — there is no LLM in the scoring path. Every score is deterministic arithmetic on structured recruiter inputs. No generated text, no inferred data, no model output. Weights are hardcoded in `app.py` and cannot drift between runs.
- Low-quality profiles are handled gracefully: unmatched skills score 0 (no noise); duration capped at 36 months (no inflation); skills score normalised by `sum(jd_weights)` so padding irrelevant skills doesn't help; missing behavioral inputs default to conservative values (RR 0.65, GitHub 0.55); unrecognised institutions score 0.55, not 0 — avoiding unfair

End-to-End Workflow

- Step 1 — Input: Recruiter opens the Gradio portal and fills each candidate's tab: name, experience, city, notice period, company, role title, institution, open-to-work, response rate, GitHub score, and skills (Name, Proficiency 1-5, Months).
- Step 2 — JD Matching: Each skill is looked up in JD_SKILLS. Matched skills get their `jd_weight`; unmatched contribute 0. Company name is checked against PRODUCT_COS / CONSULT lists; role title checked for ML/AI/NLP/LLM keywords.
- Step 3 — Dimension Scoring: `score_skills()`, `score_career()`, `score_exp()`, `score_loc()`, `score_edu()` each return 0.0–1.0. Combined: $\text{weighted_sum} = 0.35 \times \text{Skills} + 0.35 \times \text{Career} + 0.15 \times \text{Exp} + 0.10 \times \text{Loc} + 0.05 \times \text{Edu}$.
- Step 4 — Behavioral Multiplier: `beh_mult()` computes availability (45%), engagement (35%), and credibility (20%) composites, returning $\min(0.3 + 0.9 \times \text{composite}, 1.2)$.
- Step 5 — Output: `final_score = round(weighted_sum × multiplier × 100, 2)`. Sorted descending. Medal podium (🏆 🥈 🥉), per-dimension bars, full CSV for submission.

System Architecture

- Frontend Layer: Gradio (gr.Blocks) with tabbed layout — one tab per candidate. Inputs: gr.Textbox (name, company, role, institution, skills list), gr.Radio (open-to-work), gr.Slider (years of experience, notice period, response rate, GitHub score). Submit button triggers the scoring pipeline. Output rendered as Markdown with progress bars and medal podium.
- Scoring Engine: 6 pure Python functions in a single app.py (12.1 kB). Data: JD_SKILLS dict (20 skills with weights), PRODUCT_COS list (Swiggy, Razorpay, Google, Flipkart...), CONSULT list (TCS, Infosys, Accenture...), TOP_LOCS set (9 cities), EDU_TIER1 keywords. No database, no API calls, no model inference — fully stateless and deterministic.
- Deployment: HuggingFace Spaces running a Docker container (Dockerfile-based). requirements.txt contains only gradio. The app launches on port 7860, fully public. Repo: github.com/bhoomichowksey/redrob-ranker. Demo: huggingface.co/spaces/bhoomichowksey/redrob-ranker-demo.

Results & Performance

- The 3 pre-loaded demo candidates produce an intuitive ranking: Priya Sharma (Swiggy, IIT Bombay, 7 yrs, open-to-work, RR 0.92, GH 0.85) → ~96.5/100; Anita Rao (Razorpay, NIT Trichy, 8 yrs, RR 0.75) → ~88/100; Rahul Verma (TCS/consulting, state university, 3 yrs, 90-day notice, not open-to-work, RR 0.30, GH 0.20) → ~35/100. The large gap between Priya and Rahul is fully explained by visible sub-scores — consulting co_score (0.28), underexperience (0.35), long notice (0.20), and weak behavioral signals all compound. Rankings match recruiter intuition exactly.
- Scoring 5 candidates takes under 50ms — all arithmetic on Python dicts, no model inference, no network calls. Runs on the HuggingFace Spaces free tier (2 vCPU, 16 GB RAM), zero GPU required. Cold start under 10 seconds. Single dependency: gradio. Scales to hundreds of candidates by extending the DEFAULTS list.

Technologies Used

- Python 3.x — chosen for rapid development, clean arithmetic, and zero-dependency scoring logic. All 6 scoring functions are pure Python with no external ML library needed.
- Gradio (gr.Blocks) — tabbed layout lets a recruiter score up to 5 candidates side-by-side in the browser with zero frontend code. Sliders, radio buttons, and text inputs map directly to scoring inputs. Markdown output renders progress bars natively.
- Docker + HuggingFace Spaces — zero-cost, always-on public deployment. Dockerfile: Python base image + gradio. No server management needed. Live demo: huggingface.co/spaces/bhoomichowksey/redrob-ranker-demo
- GitHub — version control and public code hosting. 8 commits document the development history. The repo is fully public for judge review: github.com/bhoomichowksey/redrob-ranker

Submission Assets

 Live Demo: huggingface.co/spaces/bhoomichowksey/redrob-ranker-demo

 GitHub Repository: github.com/bhoomichowksey/redrob-ranker — Full source code (app.py, Dockerfile, requirements.txt). All scoring logic in a single 12.1 kB file. Public and ready for judge review.

 Ranked Output CSV: Generated from the HireScore portal via the Submit button. Columns: Rank, Candidate Name, Final Score, Skills, Career, Experience, Location, Education, Behavioral Multiplier.

 Approach Document: This deck (PDF + PPTX) submitted as the required approach PDF.



INDIA.RUNS

Build what next India runs on

THANK YOU